Programación con Objetos 3

Introducción a “Conceptos fundamentales de la programación con objetos”

Autores:

- Facundo Javier Gelatti — javiergelatti@gmail.com

Este documento está bajo una licencia CC BY 4.0

Índice

Introducción **1**

Sobre la diversidad de vocabulario 1

El vocabulario que elegimos usar 4

Críticas posibles a este enfoque 4

El origen del apunte para Objetos 1 5

Última modificación: 2026-03-29

Introducción

El siguiente documento forma parte del material que usamos en la materia “Programación con Objetos 1”, que elaboramos de manera conjunta con Máximo Prieto.

La idea es que puedan usarla como lectura adicional para Objetos 3, con algunos conceptos que sirven para entender el paradigma orientado a objetos. Si no quieren leer todo, pueden empezar desde la sección “Definiendo al paradigma de objetos desde sus postulados elementales”.

A continuación voy a dar contexto adicional sobre el contenido de este material, con la intención de ayudar un poco en la lectura.

Sobre la diversidad de vocabulario

En el desarrollo de software en general, y la programación orientada a objetos en particular, usamos muchas palabras. A veces no de la mejor manera, y casi nunca de manera consistente[1]. Por ejemplo, cada lenguaje tiene sus palabras específicas para referirse a las distintas partes de un objeto. También hay diferentes enfoques sobre la programación (con objetos, y en general), cada uno con sus conceptos y familias de definiciones preferidas.

Tomemos los siguientes ejemplos, que habría que leer con atención:

- En Java, una clase puede definir *campos* (*fields* en inglés), que son variables asociadas a cada instancia. Un método puede tener el mismo nombre que un campo, ambas cosas viven en espacios de nombres separados.

  Se le llama *miembros* (*members* en inglés) de una clase a los elementos que ésta declara y que participan en la herencia. Hay cuatro tipos: campos, métodos, clases anidadas e interfaces anidadas[2].

  Además, en JavaBeans, hay una convención en donde una *propiedad* es un par de métodos `getX` / `setX`.

  En el contexto de los *records*, agregados en Java 16, hablamos de *componentes* (aunque internamente se generan campos privados).

- En Python, los objetos pueden definir *atributos*, que es todo lo que se puede acceder, sintácticamente, usando un punto (ej. `obj.x`).

[1] Esto tiene que ver, entre otras cosas, con una característica de nuestra profesión, en la que construimos conceptos con grados de libertad prácticamente ilimitados. No hay nada inherente a la realidad que refute las definiciones vagas o los usos inconsistentes de nuestro vocabulario. En ese sentido, en informática tenemos que exigirnos nosotros mismos la disciplina necesaria para poder construir un cuerpo de conocimientos sólido y consistente (dentro de lo posible), lo cual no es tarea fácil.

[2] Los *constructores* no forman parte de esta lista, y por lo tanto no se heredan.

De manera predeterminada, las instancias de una clase pueden agregar atributos arbitrarios en cualquier momento. Para evitar esto, se puede usar el atributo `__slots__` de la clase, declarando de antemano todos los atributos que tendrán las instancias.

Los métodos son atributos de una clase cuyo valor es una función (que recibe como primer parámetro al objeto receptor del mensaje, lo que llamamos usualmente "self"). Por lo tanto, si quisiéramos traducir de manera directa desde Java un ejemplo de una clase con un campo y un método que tienen el mismo nombre, no podemos hacerlo. En Python, ambas cosas (el método y el campo en Java) serían atributos (de la clase y de la instancia, respectivamente). Por lo tanto, si tuvieran el mismo nombre, el de la instancia "pisaría" al de la clase, y ese objeto perdería el acceso al método.

Por otro lado, una *propiedad* es un atributo que involucra cierta lógica de acceso, creado con el decorador `@property`.

- En JavaScript, un objeto está formado por un conjunto de *propiedades*, cada una con un valor asociado.

  Un método es una propiedad cuyo valor es una función. Por lo tanto, decir "tengo un método y una propiedad que tienen el mismo nombre" es algo que, en este contexto, o bien es superfluo (el método y la propiedad son en realidad la misma cosa), o bien es necesariamente falso (a diferencia de lo que pasaría en Java, por ejemplo).

  Un *atributo* es una característica de una propiedad (ej. *Writable*, para saber si es una propiedad de sólo lectura o no). En ese sentido, aquí los atributos son como una especie de "meta-propiedad".

  Se le llama *campo* a las declaraciones sintácticas que se hacen en una clase. En el caso de que sean *campos públicos*, se crea una propiedad con el mismo nombre en cada instancia de la clase. En el caso de que sean *campos privados* (usando la sintaxis `#campo`), no son propiedades y se gestionan con un mecanismo diferente[3].

  Se le llama *accessor* a una propiedad definida mediante funciones `get` y `set` (es un concepto similar a las propiedades en Python).

  Por otro lado, *internal slot* es el nombre que usa la especificación de ECMAScript para especificar información adicional asociada a un objeto, usualmente invisible desde nuestros programas.

  Además, las *clases* en JavaScript son un concepto opcional, que se agregó en ES6 (2015), sin cambiar el modelo subyacente del lenguaje. En ese sentido, por razones históricas, en JavaScript *una clase es una función*.

[3] Se manejan internamente mediante un *slot* `[[PrivateFieldValues]]`.

- En C#, se usa la palabra *campo* de forma análoga a Java.

  Una *propiedad* es una cosa aparte, que se accede con la misma sintaxis para acceder a un campo, pero que en realidad es un par de métodos `get` y `set`. En ese sentido, una propiedad puede ser parte de una interfaz, mientras que un campo no. Una *auto-property* es una propiedad para la que el compilador genera automáticamente un campo privado y la implementación trivial de `get` y `set`.

  La especificación de C#, además, organiza todo lo que puede vivir en una clase (o, eventualmente, interfaz), y los llama *miembros* (similar a Java, aunque en este caso no implica que se hereden). Hay dos tipos: *data members* (constantes, campos, tipos anidados) y *function members* (métodos, propiedades, operadores[4], constructores, indexadores[5], eventos[6], finalizadores[7]).

- En Ruby y Smalltalk tenemos *variables de instancia* (nombre que supone la existencia de clases), en Self (donde no hay clases) tenemos *slots*, en C++ *variables miembro* (*member variables* en inglés), en Swift hay *propiedades almacenadas* y *propiedades computadas*, …

- Por último, UML usa su propia terminología, que no coincide con la de ningún lenguaje de programación.

  El término central en UML es *atributo*. Los atributos son lo que aparece en el segundo compartimiento del rectángulo que representa a una clase (el primero es el nombre, el tercero las *operaciones*[8]). Su significado coincide en este caso con el de campo en Java, o variable de instancia en Ruby/Smalltalk.

  Pero UML también usa la palabra *asociación*, que se traduce en algo exactamente igual que un atributo al momento de programar, pero que es distinto para UML. Una asociación es una línea que conecta dos clases, indicando que las instancias de una tienen una referencia a instancias de la otra.

  Ambas son formas de representar lo que UML llama *propiedad*, que es un concepto que abarca tanto a los atributos como a los extremos de una asociación.

  Finalmente, una *característica* (*feature* en inglés) es un término que agrupa tanto *características estructurales* (atributos/propiedades) como *características de comportamiento* (operaciones/métodos).

❦

[4] Los operadores son “métodos” asociados a operadores (ej. como +, *, etc.). En Ruby o Smalltalk serían simplemente métodos.
[5] Permiten usar corchetes sobre un objeto, como si fuera un array o un diccionario.
[6] Es el mecanismo de C# para el patrón observador. Permite que un objeto notifique a otros cuando algo ocurre, sin conocerlos directamente.
[7] Es lo que se ejecuta cuando el recolector de basura destruye un objeto. Es similar a los *destructores* de C++.
[8] “Operación” en UML significa algo similar a “mensaje”.

Creo que estos son ejemplos suficientes para ver que, en el contexto de los lenguajes orientados a objetos, palabras como “atributo” o “propiedad” son altamente ambiguas.

Esta es la razón por la que decidimos usar un vocabulario diferente en Objetos 1, para evitar cualquier tipo de confusión. Voy a describir ahora una parte de ese vocabulario.

El vocabulario que elegimos usar

Partimos desde una postura minimal: lo único que podemos hacer con un objeto es enviarle mensajes. Si eso es cierto, entonces la única razón que tiene un objeto para conocer a otro es para *colaborar* con él (es decir, enviarle mensajes). Por lo tanto, el rol que juega ese otro objeto, en ese contexto, es el de *colaborador*.

Pero un objeto tiene varias maneras de conocer a sus colaboradores. Pueden ser:

- *Colaboradores globales*, que son objetos bien conocidos, con los que cualquier objeto puede colaborar (por ejemplo, los booleanos o los números).
- *Colaboradores internos*, que son objetos a los que se conoce *en particular* y de manera sostenida en el tiempo. A nivel implementación, esto es equivalente a un *campo privado* en Java, a una *variable de instancia* en Ruby/Smalltalk, a un atributo/asociación en UML, etc.
- *Colaboradores externos*, que son objetos que se conocen puntualmente como parte de un mensaje. También podemos decir que son los *argumentos* del mensaje.
- *Colaboradores locales*, que son objetos que se conocen temporalmente en la activación de un método, pero que no son los argumentos del mensaje. Es lo que usualmente se conoce como *variables locales o temporales*.

Una aclaración aquí es que cuando hablamos de “colaboradores” no nos referimos a los objetos puntuales “en general”, sino a cómo un objeto se refiere a ellos en un contexto determinado. En ese sentido, un mismo objeto podría ser colaborador global, interno, externo y local al mismo tiempo.

Por otra parte, los métodos no son (necesariamente) colaboradores internos del objeto ni de la clase de la cual el objeto es instancia. Por lo tanto, el significado de “colaborador interno” es *sólo un caso particular* del de *propiedad* en JavaScript o de *atributo* en Python, etc.

Sin embargo, sí agrupamos conceptualmente a los métodos y a los colaboradores internos de un objeto como partes de su *implementación interna*.

Críticas posibles a este enfoque

Seguramente encuentres cosas en este apunte que te suenen raras o con las que no estés de acuerdo. Una posibilidad es que simplemente sea porque es un enfoque diferente, pero que vale la pena explorar (y explorar algo desconocido es algo que nos suele costar). Pero otra posibilidad es que hayas encontrado algún hueco o algo para mejorar.

La única manera de construir conocimiento es criticando lo existente, lo que justifica su modificación o reemplazo. Entonces, valoramos cualquier tipo de cuestionamiento o crítica a este contenido, siempre y cuando tenga una buena justificación. Reconocer que este enfoque es *uno de los muchos posibles* es un primer paso para habilitar la construcción de mejores enfoques. Cualquier idea que te surja, podemos charlar durante la cursada.

El origen del apunte para Objetos 1

En el año 2025 iba a sumarme a Objetos 1, como instructor suplente. A inicios de ese año empecé a escribir el documento de “Conceptos fundamentales de la programación con objetos”, para proponérselo a Máximo. La idea era documentar las ideas sobre la programación orientada a objetos que él solía dar como parte de la primera clase de la materia, titulada “Principios de Orientación a Objetos”, en un apunte que se pudiera usar para consultar y repasar.

Quienes hayan cursado con él previo al año 2025, seguramente deben recordar esa primera clase. Ésta era una de las diapositivas más icónicas:

Por razones de longitud del apunte, tuvimos que dejar de lado la conclusión de la discusión sobre llegar a una definición unificada de “programa”, y concentrarnos en lo que tiene que ver específicamente con el paradigma de objetos.

Cierro esta introducción con una frase que Máximo incluía en una de las primeras diapositivas:

> *"He intentado, no sé con qué fortuna, la redacción de cuentos directos. No me atrevo a afirmar que son sencillos; no hay en la tierra una sola página, una sola palabra, que lo sea, ya que todas postulan el universo, cuyo más notorio atributo es la complejidad...“*
>
> —Jorge Luis Borges, en el Prólogo de El Informe de Brodie